

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)*

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 50

Code of Conduct

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sandhya

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

NAME: SHIVATHMIKA V

REGNO:192521028

SUBJECT: CLOUD COMPUTING CSA 1513

CO5 AT-1

QUESTION 1 – HDFS-BASED STORAGE FOR A MEDIA PLATFORM

Problem Statement: Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient.

Understanding the Constraints

Availability must exceed 99.9%, requiring fault tolerance.

Replication factor cannot exceed 3; critical data should therefore use RF=3.

Storage must grow economically without unnecessary copies.

The platform may contain large media files, metadata, thumbnails and logs.

Proposed Architecture

Media Sources → Ingestion → HDFS DataNodes → YARN/MapReduce → Analytics

Use an Active and Standby NameNode for HDFS High Availability and multiple DataNodes for block storage.

Replication and Availability

Use replication factor 3 for critical HDFS blocks.

Enable rack-aware replica placement so copies are not concentrated in one rack.

Use DataNode heartbeats and block reports to detect failures and recreate lost replicas.

NameNode automatic failover allows the Standby to take over if the Active NameNode fails.

Cost-Efficient Growth

Use compression and efficient file formats where appropriate.

Separate frequently accessed media from archival data and move old content to lower-cost storage according to retention policy.

Scale horizontally by adding DataNodes as capacity grows instead of replacing the whole cluster.

Avoid excessive small files because they increase metadata overhead.

Constraint Satisfaction

99.9%+ availability: NameNode HA, replication, rack awareness and monitoring.

Replication ≤ 3 : RF=3 is the maximum and practical choice for critical blocks.

Cost efficiency: compression, archival policies and horizontal expansion reduce unnecessary cost.

Fault tolerance: replicas permit continued access after individual node failure

QUESTION 2 – MAPREDUCE LOG PROCESSING WITH A 20-MINUTE WINDOW

Problem Statement: Design a MapReduce workflow that processes log data within a 20-minute batch window using limited cluster nodes.

Understanding the Constraints

The complete batch must finish within 20 minutes.

Only a limited number of worker nodes are available.

Network transfer, disk I/O and excessive task creation must be minimized.

Work should be distributed evenly across the available nodes.

Proposed Workflow

Logs in HDFS → Input Splits → Mapper → Combiner → Shuffle & Sort → Reducer → HDFS Output

HDFS input splits allow independent Mapper tasks to execute in parallel.

Mapper and Combiner

The Mapper parses each record and emits useful key-value pairs, such as (status_code,1).

Filtering invalid records early reduces downstream work.

A Combiner performs local aggregation before shuffle, greatly reducing network traffic for counting workloads.

Task Distribution and Performance

Choose input splits and mapper counts according to available CPU and nodes.

Use data locality so tasks process blocks close to where they are stored.

Use a controlled number of reducers so they do not compete excessively for limited resources.

Avoid data skew because one overloaded reducer can delay the whole job.

Meeting the 20-Minute Constraint

Use balanced HDFS files and avoid small-file overhead.

Use parallel mappers and combiners.

Optimize parsing and reduce unnecessary transformations.

Measure job-stage execution times and tune mapper/reducer counts using representative workloads.

Hadoop can retry failed tasks on healthy worker nodes.

Conclusion

A data-local MapReduce design with parallel mappers, combiners, controlled reducers and balanced partitioning maximizes throughput on a small cluster and improves the probability of completing within 20 minutes.

CSA15 – Cloud Computing and Big Data Analytics | Assessment Tool 1

QUESTION 3 – YARN SCHEDULER POLICY FOR ETL, REPORTING AND ML

Problem Statement: Design a YARN scheduling policy for ETL, reporting and machine-learning jobs under constraints of fairness, ETL priority and node-failure recovery.

Understanding the Constraints

ETL must receive higher priority because downstream analytics depend on prepared data.

Reporting must retain sufficient capacity and should not be starved.

ML jobs can consume large CPU and memory resources, so resource limits are required.

Node failures must trigger recovery without permanently stopping workloads.

Proposed YARN Architecture

Applications → ResourceManager → Scheduler → NodeManagers → Containers

The ResourceManager manages cluster resources and NodeManagers manage resources on worker nodes.

Queue-Based Scheduling

Use Capacity Scheduler or Fair Scheduler with separate ETL, Reporting and ML queues.

Give ETL a higher priority and larger guaranteed capacity.

Reserve a minimum capacity for reporting.

Allow ML to use remaining resources and expand when other queues are idle.

Fairness and Priority

Set queue and user limits so one ML workload cannot monopolize the cluster.

ETL priority should be bounded by minimum capacity for reporting and ML.

Monitor queue wait time and utilization to tune allocations.

Failure Recovery

YARN monitors NodeManagers using heartbeats.

A failed node is marked unavailable and new containers are not allocated to it.

Failed tasks can be relaunched on healthy nodes.

Checkpointing or intermediate outputs can reduce the amount of work repeated after failure.

Conclusion

A queue-based YARN policy provides ETL priority while protecting reporting and ML resources. Resource limits, monitoring and automatic task recovery make the design fair, scalable and resilient.

CSA15 – Cloud Computing and Big Data Analytics | Assessment Tool 1

QUESTION 4 – ENTERPRISE INGESTION FROM SAN, NAS AND OPERATIONAL DATABASES

Problem Statement: Design an architecture to ingest SAN, NAS and operational database data while maintaining reliable backup and minimizing duplication.

Understanding the Constraints

Three heterogeneous source types must be integrated.

Backups must be reliable and recoverable.

Duplicate data should be minimized.

Incremental extraction should reduce source-system load and repeated copies.

Proposed Architecture

SAN / NAS / Operational DBs → Apache NiFi / ETL → Validation & Deduplication → HDFS → Backup/DR → Analytics

NiFi provides controlled routing, transformation, monitoring and provenance.

SAN/NAS Ingestion

Use controlled ingestion locations and record source metadata such as filename, timestamp and source system.

Checksums/content hashes can identify duplicate files before ingestion.

Validated files are written to HDFS with suitable partitioning.

Database Ingestion

Use incremental extraction or Change Data Capture where supported instead of repeatedly copying full tables.

Use timestamps, transaction logs or change indicators to identify new/modified records.

Partition HDFS output by date or business unit for efficient queries.

Backup and Reliability

HDFS replication protects against individual DataNode failure.

Maintain an additional disaster-recovery/backup strategy for critical data according to organizational requirements.

Periodically test restoration; a backup is only reliable if it can actually be recovered.

Minimizing Duplication and Quality

Use checksums and source metadata for duplicate detection.

Use a data catalog containing source, version, ownership and ingestion time.

Avoid unnecessary intermediate copies and use compression.

Validate schema, record counts, mandatory fields and file integrity; route failed records to an error path.

Conclusion

NiFi/ETL plus incremental extraction, checksum-based deduplication, HDFS replication and tested disaster recovery provides reliable enterprise ingestion while controlling storage growth and duplicate data.

CSA15 – Cloud Computing and Big Data Analytics | Assessment Tool 1

QUESTION 5 – END-TO-END SECURE HADOOP ECOSYSTEM WITH NIFI/ETL

Problem Statement: Design an end-to-end Hadoop ecosystem solution satisfying secure data movement, high availability and integration with Apache NiFi or ETL pipelines.

Understanding the Constraints

Data must move securely from enterprise sources.

Hadoop services must remain available during component or node failures.

NiFi/ETL must integrate heterogeneous sources.

The design should use HDFS, MapReduce and YARN as core Hadoop components.

End-to-End Architecture

Enterprise Sources → Apache NiFi → Secure Transfer → HDFS → YARN → MapReduce/Analytics → Results/BI

NiFi manages ingestion, routing, transformation, validation and monitoring; HDFS stores data; YARN allocates resources; MapReduce performs distributed batch processing.

Secure Data Movement

Use authenticated and encrypted connections, such as TLS, between sources and ingestion services.

Apply role-based authorization and certificate/user authentication to NiFi flows.

Encrypt sensitive data at rest in HDFS and manage keys securely.

Use secure APIs and restrict access to ingestion endpoints.

High Availability

Use Active and Standby NameNodes with automatic failover.

Use replicated HDFS blocks with rack-aware placement.

Deploy resilient YARN control components where supported.

Monitor DataNodes and NodeManagers; retry failed tasks on healthy nodes.

NiFi / ETL Integration

NiFi can ingest databases, files, APIs and IoT data.

Processors can validate, filter, transform and route data to HDFS.

FlowFile provenance records source, time and processing history.

Use incremental database extraction/CDC and checksum checks for efficient ingestion.

Hadoop Processing and Governance

HDFS should separate raw, processed and curated datasets.

YARN queues can control resources among ETL, analytics and reporting.

MapReduce uses HDFS data locality for distributed processing.

Audit logs, permissions, encryption and monitoring support governance and security.

Conclusion

The combined NiFi–HDFS–YARN–MapReduce architecture provides secure movement, highly available storage and resource-managed distributed processing while maintaining data provenance and operational monitoring